

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I M.VINAY(192311146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

M.VINAY

Signature of the student

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Set 3

1. Autonomous Drone Delivery System

A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.

Tasks:

- (i) Design intermediate code for drone navigation and obstacle detection logic. (K3)
- (ii) Explain how Boolean expressions support autonomous decision-making in drones. (K4)
- (iii) Analyze the importance of compiler optimization in real-time delivery systems. (K5)

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The compiler processes declarations, assignment statements, case statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

- (i) Construct intermediate code for reservation and scheduling operations. (K3)
- (ii) Discuss how case statements efficiently handle passenger ticket categories. (K4)
- (iii) Evaluate the role of compiler optimization in large-scale transportation systems. (K5)

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the

embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

- (i) Generate intermediate code for load balancing and fault detection logic. (K3)
- (ii) Explain the significance of backpatching in smart energy management systems. (K4)
- (iii) Analyze how compiler optimization improves the performance of smart grids. (K5)

4. AI-Based Fraud Detection System

A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.

Tasks:

- (i) Develop intermediate code for fraud detection and alert generation logic. (K3)
- (ii) Explain how procedure calls improve modularity in fraud detection systems. (K4)
- (iii) Evaluate the role of compiler design in secure financial applications. (K5)

5. Smart Warehouse Inventory Management System

A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.

Tasks:

- (i) Design intermediate code for inventory tracking and stock update operations. (K3)
- (ii) Explain how case statements help manage multiple product categories efficiently. (K4)
- (iii) Analyze the importance of compiler optimization in warehouse automation systems. (K5)

ANSWERS:

1. Autonomous Drone Delivery System

(i) Intermediate code for drone navigation and obstacle detection — K3

Assume:

- distance = distance to destination
- obstacle = obstacle detected
- battery = battery percentage
- speed = drone speed

1. if battery < 20 goto L1
2. if obstacle == true goto L2
3. if distance > 0 goto L3
4. goto L4

L1:

5. action = "Emergency Landing"
6. goto L4

L2:

7. speed = 0
8. action = "Avoid Obstacle"
9. goto L4

L3:

10. speed = 40
11. action = "Move Forward"

L4:

12. stop

This intermediate code allows the drone to check **battery level, obstacles, and navigation conditions** before selecting an action.

(ii) Boolean expressions in autonomous decision-making — K4

Boolean expressions produce true or false results and allow the drone to make decisions automatically.

Examples:

obstacle == true

battery < 20

distance > 0

battery > 20 AND obstacle == false

For example, the condition:

if battery > 20 AND obstacle == false

allows the drone to continue flying only when sufficient battery is available and the path is clear. Thus, Boolean expressions support **navigation, collision avoidance, battery monitoring, emergency landing, and route selection.**

(iii) Importance of compiler optimization — K5

Compiler optimization is important because autonomous drones operate under **real-time constraints**. Optimized intermediate code reduces unnecessary instructions and improves execution speed. Faster execution enables quicker obstacle detection and route adjustment.

It also reduces CPU and memory usage, which can lower power consumption. Most importantly, optimized code improves the reliability of safety-critical operations such as **collision avoidance and emergency landing.**

2. Railway Reservation and Scheduling System

(i) Intermediate code for reservation and scheduling — K3

Assume:

- availableSeats = available seats
- request = number of requested seats
- fare = ticket fare

1. if availableSeats >= request goto L1
2. goto L2

L1:

3. availableSeats = availableSeats - request
4. status = "Confirmed"
5. fare = request * 500
6. call updateReservation
7. goto L3

L2:

8. status = "Waiting List"
9. call updateWaitingList

L3:

10. call displayStatus

This code checks seat availability, performs seat allocation, calculates the fare, and updates the passenger's reservation status.

(ii) Case statements for passenger ticket categories — K4

A case statement efficiently selects an operation based on the passenger category.
case category of

- 1: fare = baseFare
- 2: fare = baseFare * 0.50

3: fare = baseFare * 0.75

4: fare = baseFare * 0.60

end case

For example:

1 → General

2 → Senior Citizen

3 → Student

4 → Child

Instead of writing many nested if-else statements, a case statement provides a **clear and structured selection mechanism**. It makes ticket-category processing easier to maintain and extend.

(iii) Compiler optimization in large-scale transportation systems — K5

Railway systems may process thousands of reservations simultaneously. Compiler optimization reduces the number of instructions required for seat allocation, cancellation, fare calculation, and scheduling.

It improves **response time, memory utilization and transaction throughput**. During peak booking periods, optimized code helps prevent unnecessary processing delays and allows the reservation system to handle more users efficiently.

3. Intelligent Energy Management System

(i) Intermediate code for load balancing and fault detection — K3

Assume:

- load = current electricity load
- maxLoad = maximum safe load
- fault = fault status

1. if fault == true goto L1

2. if load > maxLoad goto L2

3. goto L3

L1:

4. supply = 0

5. action = "Isolate Fault"

6. goto L4

L2:

7. excess = load - maxLoad

8. call redistributeLoad

9. action = "Load Balanced"

10. goto L4

L3:

11. action = "Normal Operation"

L4:

12. call updateGridStatus

The code first checks for faults and then determines whether load redistribution is required.

(ii) Significance of backpatching — K4

Backpatching is a compiler technique used to fill in the target addresses of jump instructions after those target locations become known.

For example, initially:

if fault == true goto _

if load > maxLoad goto _

The jump destinations are unknown. After generating the remaining code, the compiler fills them:

if fault == true goto L1

if load > maxLoad goto L2

Backpatching is useful for translating **Boolean expressions, conditional statements and control-flow structures**. In a smart grid, efficient backpatching helps generate correct control flow for fault detection and load redistribution.

(iii) Compiler optimization in smart grids — K5

Smart grids require rapid processing because power demand and fault conditions can change quickly. Compiler optimization reduces redundant calculations and unnecessary memory operations.

This enables faster **fault detection, load balancing, power redistribution and system recovery**. Efficient code can also reduce the computational resources required by embedded controllers. Therefore, optimization contributes to the **reliability, responsiveness and energy efficiency** of smart-grid systems.

4. AI-Based Fraud Detection System

(i) Intermediate code for fraud detection and alert generation — K3

Assume:

- amount = transaction amount
- limit = transaction threshold
- locationMismatch = unusual transaction location

1. if amount > limit goto L1

2. if locationMismatch == true goto L1

3. goto L2

L1:

4. status = "Suspicious"

5. call generateAlert

6. call blockTransaction

7. goto L3

L2:

8. status = "Approved"

9. call processTransaction

L3:

10. call updateTransactionLog

The intermediate code evaluates suspicious conditions and triggers fraud-control procedures when necessary.

(ii) Procedure calls and modularity — K4

Procedure calls divide a large fraud-detection program into smaller reusable modules.

Examples include:

call checkTransaction

call calculateRiskScore

call generateAlert

call blockTransaction

call updateTransactionLog

Each procedure performs a specific task. This improves **modularity, readability, testing, debugging and code reuse**.

For example, generateAlert() can be called from multiple fraud rules instead of rewriting the alert-generation code each time. Changes to alert logic can then be made within a single procedure.

(iii) Compiler design in secure financial applications — K5

Efficient compiler design helps financial applications process large numbers of transactions quickly. Optimized code can reduce fraud-detection latency and improve transaction-processing throughput.

A well-designed compiler also generates predictable and correct machine code, supports safe handling of procedure calls and memory, and can enable security protections during compilation. This contributes to **performance, reliability and security** in financial applications.

5. Smart Warehouse Inventory Management System

(i) Intermediate code for inventory tracking and stock updates — K3

Assume:

- stock = current stock
- soldQty = quantity sold
- reorderLevel = minimum stock level

1. stock = stock - soldQty

2. if stock <= reorderLevel goto L1

3. goto L2

L1:

```
4. status = "Low Stock"
5. reorderQty = 100 - stock
6. call placeOrder
7. goto L3
```

L2:

```
8. status = "Stock Available"
```

L3:

```
9. call updateInventory
10. call displayStock
```

This code updates stock after a sale and automatically initiates reordering when inventory falls below the minimum level.

(ii) Case statements for multiple product categories — K4

A warehouse may contain categories such as electronics, food, clothing and furniture. A case statement can select the appropriate handling operation.

case productCategory of

```
1: call handleElectronics
2: call handleFood
3: call handleClothing
4: call handleFurniture
```

end case

For example:

```
1 → Electronics
2 → Food
3 → Clothing
4 → Furniture
```

Case statements make category-based processing more organized than using many nested if-else conditions. New categories can also be added easily, improving **maintainability and scalability**.

(iii) Importance of compiler optimization in warehouse automation — K5

Warehouse automation involves continuous inventory updates, barcode/RFID processing, order fulfillment and automated storage operations. Inefficient code can cause delays and synchronization problems.

Compiler optimization removes redundant instructions, improves execution speed, reduces memory usage and makes better use of processor resources. During seasonal sales or high-volume periods, this enables faster stock updates and order processing.

Therefore, compiler optimization improves **inventory accuracy, synchronization, system responsiveness and overall warehouse efficiency**.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

